

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«Пермский государственный национальный
исследовательский университет»

Институт компьютерных наук и
технологий

ОТЧЁТ

по индивидуальному заданию «динамические структуры данных»
по дисциплине «Язык программирования Python»
Вариант 13

Работу выполнил
студент группы ИТ-9 1 курса
Федоров.Д.А.
19.06.2026 г.

Работу проверила
Шилина.А.В.
19.06.2026 г.

Пермь 2026

Содержание

Постановка задачи.....	3
Алгоритм решения.....	4
Тестирование.....	7
Код программы.....	11

Постановка задачи

Написать программу, которая по заданной формуле строит дерево и производит вычисления с помощью построенного дерева. Формула задана в традиционной инфиксной записи, в ней могут быть скобки, максимальная степень вложенности которых ограничивается числом 10. Аргументами могут быть целые числа и переменные, задаваемые однобуквенными именами. Допустимые операции: +, -, *, /. Унарный минус допустим. С помощью построенного дерева формулы упростить формулу, заменяя в ней все поддеревья, соответствующие формулам $(f_1 f_3 \pm f_2 f_3)$ и $(f_1 f_2 \pm f_1 * f_3)$ на поддеревья, соответствующие формулам $((f_1 \pm f_2) f_3)$ и $(f_1 (f_2 \pm f_3))$.

Алгоритм решения

Типы данных:

1. Целочисленный тип данных (int), для работы с базовыми математическими операциями.
2. Вещественный тип данных (float), для работы с дробными числами и промежуточными результатами вычислений.
3. Строковый тип данных (str), для общения с пользователем и обработки ввода.
4. Списки (list), для временного хранения токенов и очереди операторов при переводе в постфиксную запись.
5. Множества (set), для хранения уникальных переменных, обнаруженных в выражении.
6. Узлы (CalcNode), самописный класс, элементарный узел двоичного дерева. Хранит значение (число, переменная или оператор) и ссылки на левое и правое поддерево.
7. Дерево выражений (CalcTree), самописный класс. Управляет деревом узлов CalcNode, обеспечивая построение, упрощение, вычисление и обход.
8. Стек (Stack), самописный класс на основе связанного списка узлов StackNode. Реализует принцип LIFO и используется внутри алгоритма Дейкстры для хранения операторов и поддеревьев.

Описание алгоритма:

Алгоритм работы программы состоит из четырёх основных этапов:

Этап 1. Токенизация входной строки

1. Входная строка очищается от пробелов.
2. Идём по строке. Цифры и буквы собираются в токен. Операции +, -, *, / и скобки — отдельные токены.
3. Унарный минус определяется по контексту: если «-» стоит в начале выражения или сразу после +, -, *, / или (, он заменяется специальным маркером «~».

Этап 2. Перевод в постфиксную запись (алгоритм Дейкстры)

1. Для каждого токена: числа/переменные напрямую попадают в очередь вывода; операции выталкиваются с учётом приоритета (* и / выше, чем + и -; ~ выше всех), «(» помещается в стек, если встречаем «)» выталкиваем всё до парной «(».
2. После обхода всех токенов остаток стека операторов выталкивается в очередь вывода.

Этап 3. Построение дерева выражений из постфиксной записи

1. Для каждого токена из постфиксной записи: если токен «число/переменная» — создаётся листовой узел CalcNode и помещается в стек деревьев.

2. Если токен «операция» (включая $\sim$): с вершины стека деревьев снимаются ветви (1 для $\sim$ и 2 для бинарных), создаётся узел CalcNode с оператором и помещается обратно в стек. Унарный $\sim$ представляется как «0 - X(узел)».
3. В результате в стеке остаётся один узел — корень дерева выражения.

Этап 4. Алгебраическое упрощение

Обход дерева производится рекурсивно снизу вверх (postfix). Проверяются четыре шаблона:

1. **Шаблон 1:** $(f1 \cdot f3) \pm (f2 \cdot f3) \Rightarrow ((f1 \pm f2) \cdot f3)$ — если правые поддеревья умножений равны, общий множитель выносится вправо за скобки.
2. **Шаблон 2:** $(f1 \cdot f2) \pm (f1 \cdot f3) \Rightarrow (f1 \cdot (f2 \pm f3))$ — если левые поддеревья умножений равны, общий множитель выносится влево за скобки.
3. **Шаблон 3:** $(f1 \cdot f2) \pm (f3 \cdot f1) \Rightarrow (f1 \cdot (f2 \pm f3))$ — перекрестное равенство (левое первого и правое второго поддерева).
4. **Шаблон 4:** $(f2 \cdot f1) \pm (f1 \cdot f3) \Rightarrow (f1 \cdot (f2 \pm f3))$ — перекрестное равенство (правое первого и левое второго поддерева).

Упрощение применяется рекурсивно до тех пор, пока ни один из шаблонов не срабатывает — таким образом, выражение вида $x * y + x * z$ сводится к $x * (y + z)$.

Архитектура программы и описание модулей:

1. Модуль структур данных (objects.py)

Данный модуль отвечает за реализацию структур данных и инкапсуляцию их внутренних свойств.

Класс CalcNode:

Представляет собой элементарный узел двоичного дерева. Хранит значение (data) и ссылки на левое (left) и правое (right) поддерева.

Класс CalcTree:

Управляет деревом выражения. Реализует методы:

- `insert_infix(expression)` — парсинг инфиксного выражения и построение дерева через алгоритм Дейкстры.
- `simplify()` — рекурсивный обход дерева с выносом общего множителя за скобки.
- `evaluate(variables)` — числовое вычисление по дереву.
- `get_variables()` — поиск всех переменных в дереве.
- `to_string()` — вывод формулы из дерева в инфиксной записи.
- `copy()` / `copy_s()` — глубокое копирование дерева.
- `is_equal()` — рекурсивное сравнение двух поддеревьев.

Классы Stack и StackNode:

Классы Stack и StackNode реализуют стек на основе связного списка с методами `add()` и `pop()`. Используются внутри алгоритма Дейкстры для хранения операторов и промежуточных поддеревьев.

2. Модуль интерфейса (main.py)

Модуль отвечает за взаимодействие с пользователем, текстовое меню и защиту от некорректного ввода.

- Цикл управления: бесконечный цикл с текстовым меню, обеспечивает возврат после каждой операции.
- Валидация: проверка опции меню, пустоты формулы, баланса скобок и степени вложенности (≤ 10).
- Ввод переменных: интерактивный запрос на числовые значения для переменных с обработкой ValueError.

Тестирование

Входные данные содержат только полезную информацию. Например, ввод единицы для того, чтобы зайти в программу абсолютно бесполезная информация для тестов.

Блок 1. Тестирование основных ситуаций.

Тест 1. Формула без переменных.

Входные данные:

Формула: $2 + 3 * 4$

Выходные данные:

Дерево исходной формулы успешно построено: $(2 + (3 * 4))$

Формула после алгебраического упрощения по дереву: $(2 + (3 * 4))$

Результат вычисления по дереву: 14.0

Вывод: Тест пройден успешно.

Тест 2. Вынос общего множителя.

Входные данные:

Формула: $x * y + x * z$

Выходные данные:

Дерево исходной формулы успешно построено: $((x * y) + (x * z))$

Формула после алгебраического упрощения по дереву: $(x * (y + z))$

Введите значение для переменной x: 3

Введите значение для переменной y: 4

Введите значение для переменной z: 5

Результат вычисления по дереву: 27.0

Вывод: Тест пройден успешно.

Тест 3. Унарный минус.

Входные данные:

Формула: $-x + 3$

Выходные данные:

Дерево исходной формулы успешно построено: $((-x) + 3)$

Формула после алгебраического упрощения по дереву: $((-x) + 3)$

Введите значение для переменной x: 5

Результат вычисления по дереву: -2.0

Вывод: Тест пройден успешно.

Тест 4. Деление на ноль.

Входные данные:

Формула: $5 / (3 - 3)$

Выходные данные:

```
=====
| Критическая ошибка: Обнаружено деление на ноль при расчете! |
=====
```

Рис. 1. Ошибка деления на 0.

Вывод: Тест пройден успешно.

Блок 2. Тестирование ошибок пользовательского ввода.

Тест 5. Ввод неверной опции в главном меню.

Входные данные:

Ваш выбор: apple

Выходные данные:

```
=====
| Неверная опция, перезапуск программы |
=====
```

Рис. 2. Ошибка ввода 1.

Результат: Программа загрузила главное меню заново.

Вывод: Тест пройден успешно.

Тест 6. Пустая строка формулы.

Входные данные:

Формула: «Пустое поле»

Выходные данные:

```
=====
| Строка формулы не может быть пустой, перезапуск программы |
=====
```

Рис. 3. Ошибка ввода 2.

Результат: Программа выкинула пользователя в главное меню.

Вывод: Тест пройден успешно.

Тест 7. Нарушение баланса скобок.

Входные данные:

Формула: (x + y

Выходные данные:

```
=====
| Ошибка: Нарушен баланс скобок, перезапуск программы |
=====
```

Рис. 4. Ошибка ввода 3.

Результат: Программа выкинула пользователя в главное меню.

Вывод: Тест пройден успешно.

Тест 8. Глубина вложенности скобок превышает 10.

Входные данные:

Формула: ((((((((((x))))))))))

Выходные данные:

```
=====
| Ошибка: Степень вложенности скобок превышает 10! |
=====
```

Рис. 5. Ошибка превышения вложенности.

Результат: Программа выкинула пользователя в главное меню.

Вывод: Тест пройден успешно.

Тест 9. Неверное значение переменной.

Входные данные:

Формула: $x + y$

Введите значение для переменной x: abc

Выходные данные:

```
> Введите значение для переменной 'x': abc
| Ошибка! Значение должно быть числом. Попробуйте еще раз.
```

Рис. 6. Ошибка ввода 4.

Результат: Программа повторно запросила значение.

Вывод: Тест пройден успешно.

Дополнительные скриншоты для раздела тестирования:

```

//-----\
\\      \\      _____| n n l z
\ \     //      \ `  ≡ _ω / ~ (Спасибо что используете tree calc!)
  \ \  //      )      /
    \ \ /      )      /
      \ \ /____ \ ____/
        - \ _____) _____)

=====
| Меню программы tree calc. Пожалуйста, выберите опцию! |
=====
1) Ввести формулу и произвести вычисления
2) Выйти из программы
Ваш выбор:
```

Рис. 7. Главное меню.

```

=====
| Введите математическую формулу в инфиксной записи. |
| Допускаются: целые числа, однобуквенные переменные, |
| знаки +, -, *, /, унарные минусы и скобки (вложенность <=10). |
| Пример: x * y + x * z |
=====
> Формула: x + z * z + y

> Дерево исходной формулы успешно построено:
  ((x + (z * z)) + y)

> Формула после алгебраического упрощения по дереву:
  ((x + (z * z)) + y)

=====
| Программа обнаружила переменные. Введите их числовые |
| значения для производства вычислений. |
=====
> Введите значение для переменной 'x': 3
> Введите значение для переменной 'y': 4
> Введите значение для переменной 'z': 5

=====
| Результат вычисления по дереву: 32.0 |
=====

```

Рис. 8. Поле ввода значений.

Код программы

Код программы находится в моём репозитории на GitHub.

Ссылка на репозиторий: https://github.com/BromFD/container_sort